

Documentation technique - Lyubot V2

1. Vue d'ensemble

Lyubot V2 est un bot Node.js (CommonJS) base sur Discord.js v14 avec une integration Twitch (Helix + IRC via tmi.js).

Objectifs techniques principaux:

- centraliser des commandes de moderation Discord;
- exposer des actions Twitch depuis Discord;
- automatiser des workflows communautaires (tickets, salons vocaux temporaires, welcome, logs);
- conserver une architecture simple par dossiers (commands, handlers, services, utils, data).

2. Stack et prerequis

- Runtime: Node.js >= 20
- Langage: JavaScript (CommonJS)
- Librairies principales:
 - discord.js ^14.25.1
 - axios ^1.13.2
 - tmi.js ^1.8.5
 - dotenv ^17.2.3
 - node-cron ^4.2.1
- Dev: nodemon ^3.1.11

Scripts npm:

- `npm start` -> demarrage production/local
- `npm run dev` -> demarrage dev avec `nodemon` (ignore `data/**`)

3. Architecture du projet

3.1 Point d'entree

- `index.js`
 - charge `.env`;
 - instancie le client Discord avec intents/partials;
 - initialise les handlers;
 - configure la presence;
 - demarre le relay Twitch chat (`startTwitchRelay`);
 - lance `client.login(DISCORD_TOKEN)`.

3.2 Dossiers

- `commands/`: slash commands (fichiers simples ou tableaux de commandes exportees)
- `handlers/`: branchent les listeners sur le client
- `services/`: logique metier reutilisable (ticket, Twitch API/chat, logs)

- `utils/`: fonctions transverses (permissions, guards, pagination, temps)
- `data/`: persistance locale JSON

3.3 Flux de démarrage

1. Chargement des variables d'environnement.
2. Creation du client Discord.
3. Enregistrement du handler voix temporaire (`voiceTemp`).
4. Chargement recursif des commandes via `handlers/commandHandler.js`.
5. Enregistrement des listeners de logs.
6. `ClientReady`: presence + lancement du relay Twitch IRC.

4. Gestion des commandes

`handlers/commandHandler.js`:

- scanne recursivement `commands/**/*.js`;
- accepte un export unique (`{ data, execute }`) ou un tableau de commandes;
- stocke les commandes dans `client.commands`;
- execute uniquement les interactions de type slash (`isChatInputCommand`);
- applique des regles via `meta` (`guildOnly`, `adminOnly`);
- peut redeployer les slash commands sur guilde si `CLEAN_COMMANDS_ON_START=true`.

Remarques:

- les erreurs d'execution de commande sont catch et repondent "Erreur interne.";
- la verification admin est geree par `utils/permissions.js`.

5. Modules fonctionnels

5.1 Moderation Discord

Commandes dediees:

- `discordban`, `kick`, `mute`, `unmute`, `unban`, `userinfo`, `clear`.

Permissions:

- utilise permissions natives Discord (`BanMembers`, `KickMembers`, `ModerateMembers`, `ManageMessages`)
+ controles additionnels possibles via `meta`.

5.2 Voice temporaire

`handlers/voiceTemp.js`:

- surveille `voiceStateUpdate`;
- cree un salon vocal temporaire quand un membre rejoint `CREATE_VOICE_CHANNEL_ID`;
- enregistre propriétaire + metadata dans `data/voiceRooms.json`;
- transfere la propriete automatiquement si le propriétaire quitte et qu'il reste des membres;
- supprime le salon si vide (delete differe pour eviter les races).

Commandes vocales associees:

- `rename`, `limit`, `lock`, `unlock`, `transfer`, `lead` (suivant les fichiers charges).

5.3 Tickets

Composants:

- commande `ticketpanel` pour publier un embed + bouton d'ouverture;
- `services/ticketService.js` pour creation/fermeture;
- logique de permissions staff via `ADMIN_ROLE_ID` ou `MODERATOR_ROLE_ID`.

5.4 Welcome

Commande `welcome` avec sous-commandes:

- `toggle`, `add`, `remove`, `test`.

Stockage:

- version en memoire locale de messages;
- un fichier `data/welcome.json` est present dans le repo (selon implementation active).

5.5 Twitch Helix (API)

`services/twitch.js`:

- construit les headers Helix;
- resolve l'ID broadcaster a partir de `STREAMER_USERNAME`;
- utilitaire de lookup utilisateur.

Commandes Twitch:

- recherche: `twitchsearch`
- moderation: `twitchaddmod`, `twitchremovemod`, `twitchban`, `twitchtimeout`, `twitchunban`
- bans listes: `twitchbans`, `twitchlistbans`
- roles/vips: `addmodo`, `removemodo`, `addvip`, `removevip`, `listmods`, `listvips`

5.6 Twitch chat relay

`services/twitchChat.js`:

- connexion IRC Twitch via `tmi.js`;
- relay des messages Twitch vers un salon Discord;
- mapping interne pour supprimer les messages Discord lors de `messagedeleted`, `ban`, `timeout`, `clearchat`;
- ajout de badges (modo, vip, fondateur, sub custom).

5.7 Logs serveur

`handlers/logEvents.js`:

- journalise joins/leaves membres;

- journalise activite vocale;
- journalise create/update/delete channels et roles;
- envoi d'embeds vers `LOGS_CHANNEL_ID`.

6. Variables d'environnement

6.1 Obligatoires minimales

Variable	Description
<code>DISCORD_TOKEN</code>	Token du bot Discord

6.2 Discord commandes/deploiement

Variable	Description
<code>DISCORD_CLIENT_ID</code>	ID application Discord pour deploy slash
<code>GUILD_ID</code>	Guilde cible de deploy des commandes
<code>CLEAN_COMMANDS_ON_START</code>	Si <code>true</code> , redéploie les commandes au démarrage
<code>OWNER_ID</code>	ID propriétaire pour certaines commandes admin
<code>ADMIN_ROLE_ID</code>	Role admin interne pour controles supplementaires
<code>MODERATOR_ROLE_ID</code>	Role moderation pour tickets/controles

6.3 Voice/Tickets/Welcome/Logs

Variable	Description
<code>CREATE_VOICE_CHANNEL_ID</code>	Salon vocal "create" declencheur
<code>VOICE_CATEGORY_ID</code>	Categorie de creation des salons temporaires
<code>VOICE_NAME_TEMPLATE</code>	Template nom salon (ex: 🗣️ {user})
<code>WELCOME_CHANNEL_ID</code>	Salon cible pour test welcome
<code>LOGS_CHANNEL_ID</code>	Salon texte de journalisation
<code>LOG_DIR</code>	Repertoire des logs fichiers (service logger)
<code>LOG_DISCORD_MAX_BYTES</code>	Taille max piece jointe logs

6.4 Twitch API/Relay

Variable	Description
<code>TWITCH_CLIENT_ID</code>	Client ID Helix
<code>TWITCH_USER_TOKEN</code>	User access token Helix
<code>STREAMER_USERNAME</code>	Login de la chaine cible

Variable	Description
<code>TWITCH_BOT_USERNAME</code>	Username du bot IRC Twitch
<code>TWITCH_BOT_TOKEN</code>	OAuth token IRC (<code>oauth:...</code>)
<code>TWITCH_CHANNELS</code>	Liste CSV de channels Twitch a relayer
<code>TWITCH_RELAY_CHANNEL_ID</code>	Salon Discord recevant le relay
<code>TWITCH_REMOVE_FORMAT</code>	Nettoie le format <code>/me</code> si <code>true</code>

6.5 Variables secondaires identifiees

Variable	Description
<code>STAFF_GUILD_ID</code>	Guilde staff (modules alternatifs)
<code>COMMUNITY_GUILD_ID</code>	Guilde communautaire (modules alternatifs)
<code>DEBUG</code>	Active certains logs debug

7. Donnees persistantes

Fichiers JSON:

- `data/voiceRooms.json`: map des salons vocaux temporaires et proprietaires.
- `data/welcome.json`: configuration messages welcome (si utilisee par la commande active).

Bonnes pratiques:

- ne pas versionner des donnees runtime sensibles;
- sauvegarder ces fichiers avant migration/maintenance;
- prevoir une validation JSON au chargement (deja partiellement faite sur `voiceRooms`).

8. Exploitation et operations

8.1 Installation

1. `npm install`
2. definir les variables d'environnement
3. `npm start`

8.2 Developpement

1. `npm install`
2. `npm run dev`
3. verifier la connexion Discord et la presence des slash commands

8.3 Verification rapide post-demarrage

Checklist:

- le bot est online et visible;
- la commande `/systemcheck` repond;
- les commandes Twitch repondent sans erreur de config;
- l'event de log ecrit bien dans `LOGS_CHANNEL_ID`;
- la creation de salon vocal temporaire fonctionne.

9. Risques techniques et points d'attention

- le chargement recursif peut inclure des commandes en doublon (nom identique dans plusieurs fichiers); la derniere chargee ecrase la precedente dans `client.commands`.
- la base est orientee "best effort" sur plusieurs actions (`catch(()=>{})`), ce qui evite les crashes mais peut masquer certaines erreurs operationnelles.
- la configuration Twitch depend d'un token Helix valide et scopes adaptes; sans scopes, certaines commandes renverront des erreurs API.

10. Recommandations d'amelioration

- ajouter une validation stricte de configuration au demarrage (schema env).
- normaliser les commandes pour eviter les doublons de noms.
- centraliser la gestion des erreurs (logger + codes).
- ajouter des tests de non-regression pour commandes critiques (moderation/twitch).
- ajouter un `.env.example` complet et maintenu.

11. Commandes utiles

```
npm install
npm run dev
npm start
```